

Instituto Tecnológico de Buenos Aires (ITBA)



22.07 - Programacion I

Resumen de C

Ignacio Sammartino

Índice

Names in C	3	Keyword static	18
Function that exits a program - exit()	3	Functions	18
printf and scanf	3	Functions for manipulating strings and general functions	19
Scaler types	4	<stdio.h>	19
Primitivos - Enteros	4	<stdlib.h>	19
Primitivos - Punto Flotante	5	<string.h>	19
Casteo (Implicitos)	5	Functions for manipulating files	20
Notación Científica	5	Open a file with fopen	20
Define	5	Read and write from / to a file	21
Enumeradores	5	Function pointers	22
Logica Booleana	5	Threads	23
TypeDef's	6	Concurrencia vs Paralelismo	23
Multidimensional arrays	6	Procesos vs Threads	23
Pointers	7	Pthreads API	23
Punteros y arreglos	7	Sincronización	24
Aritmetica de punteros	8	Mutex (Mutual Exclusion)	24
Equivalencias	8	Semáforos	24
Strings	8	Callbacks	24
Passing Pointers to functions arguments	9	Tipos de Invocación	24
Array of pointers	9	Aplicaciones Comunes	24
Punteros de Punteros	9	Algoritmo qsort()	24
Punteros de Constantes	9	Programación orientada a eventos	25
True vs False	10	Implementación con Punteros a Función	25
Logical operators	10	Multiple files	26
Switch	10	From Code to executable	27
While Block	10	1. Preprocesamiento (Preprocessing)	27
While	10	2. Compilación (Compilation)	27
do while	11	3. Ensamblado (Assemble)	27
For Block	11	4. Vinculación (Linking)	27
Break	12	MakeFile	28
Continue	12	HelloMake example	28
Comparing floating point values	12		
Bit operators	13		
sizeof operator	13		
Structures	14		
Nested Structures	15		
Structure operators	16		
Bitfields	16		
Unions	16		
Dynamic memory allocation on the Heap	16		
malloc	17		
calloc	17		
realloc	17		

Names in C

En el lenguaje C, los identificadores (nombres de variables, funciones, etc.) deben seguir reglas específicas:

- Son **case sensitive** (distinguen entre mayúsculas y minúsculas).
- Deben comenzar con una **letra** o un **guion bajo** («_»).
- Solo pueden contener **guiones bajos, letras o números**.

```
int variableValida;  
int _contador;  
int valor2;  
// int 2valor; // Esto causaría un error de compilació
```

Function that exits a program - exit()

exit() La función exit() permite terminar la ejecución de un programa de forma inmediata desde cualquier punto. Se encuentra definida en la librería <stdlib.h>. Comúnmente se utiliza el valor 0 para una terminación exitosa y valores distintos para indicar errores.

```
#include <stdlib.h>  
  
int main(void){  
    // Exit successfully. The program terminates at this point.  
    exit(0);  
  
    // Exit with error.  
    exit(-1);  
}
```

printf and scanf

Estas son las funciones fundamentales para la entrada y salida de datos en consola, incluidas en la librería <stdio.h>.

- printf(): Se utiliza para imprimir texto y variables con formato.
- scanf(): Se utiliza para leer datos del usuario desde el teclado.

```
#include <stdlib.h>  
  
int main() {  
    int edad;  
    printf("Ingrese su edad: ");  
    scanf("%d", &edad); // El símbolo & es necesario para obtener la dirección  
    printf("Su edad es: %d años\n", edad);  
    return 0;  
}
```

De querer escanear otro tipo de input para `scanf()`, tengo las siguientes opciones:

- `%d` : integer
- `%f` : floating point
- `%c` : char
- `%s` : NULL or “0” terminated string
- `%x` : Hexadecimal number
- `%o` : Octal number

Scaler types

Los tipos escalares en C representan valores individuales y se dividen en varias categorías según las fuentes:

- **Tipos primitivos:** Incluyen los tipos básicos como `int`, `char`, `float` y `double`.
- **Notación científica:** C permite representar números reales usando notación científica (ej. `1.2e3`).
- **C99 - `bool`:** A partir del estándar C99, se incluye el tipo booleano mediante la librería `<stdbool.h>`.
- **Enums:** Permiten definir tipos de enumeración para manejar constantes relacionadas.

```
#include <stdio.h>
#include <stdbool.h>

enum dia_semana {LUNES, MARTES, MIERCOLES};

int main() {
    int entero = 10;
    float pi = 3.14159;
    double cientifica = 1.2e-3;
    bool esVerdad = true;
    enum dia_semana hoy = LUNES;

    return 0;
}
```

Primitivos - Enteros

type	size (bits/bytes)	Range
unsigned char	8 / 1	0 to 255
char	8 / 1	-128 to 127
unsigned int	16 / 2	0 to 63535
short int	16 / 2	-32768 to 32767
int	16 / 2	-32768 to 32768
unsigned long	32 / 4	0 to 4294967295
long	32 / 4	-2147483648 to 2147483647

Primitivos - Punto Flotante

type	size (bits/bytes)	Exponente	Mantissa
float	32 / 4	8	23
double	64 / 8	11	52

Casteo (Implicitos)

Esto ocurre cuando se le asigna un valor a una variable, y el tipo de la constante no coincide con el de la variable. En dicho caso, el tipo de la constante es casteado (promovido truncado) al tipo de la variable

float	x = 3;	//almacena el 3.0	C
int	i = 8.452	//almacena el número 8	
char	a = 200	//(número negativo)	
unsigned char	b = -12	//(número positivo)	
int	c = -12.89	//(almacena el -12)	
int	e = '5'	//(almacena el ASCII del 5)	

Notación Científica

// Notacion Cientifica	C
float num_2 = 5e-5;	
double num_3 = 3.7e12;	

Define

La directiva #define que nos permite asignarle a un símbolo un valor

# define NUM_A 5.5	/* Double constant	*/	C
# define NUM_B 5.5f	/* Float constant	*/	
# define NUM_C 5.5e6L	/* Long double constant	*/	
# define ADDRESS 0x20004000UL	/* Unsigned Long constant	*/	

Enumeradores

Por default, LUNES arranca en 0

enum dias { LUNES, MARTES, MIÉRCOLES, JUEVES, VIERNES, SÁBADO, DOMINGO }	C
--	---

De queres forzar a que arranque desde otro numero.

enum dias { LUNES = 1, MARTES, MIÉRCOLES, JUEVES, VIERNES, SÁBADO, DOMINGO }	C
--	---

Logica Booleana

#include <stdbool.h>	C
bool flag = false;	
flag = true;	
if (flag){ print("Flag is true!\n");}	

TypeDef's

La palabra clave typedef se utiliza para crear un alias o un nuevo nombre para un tipo de dato ya existente, lo que mejora la legibilidad del código.

```
typedef unsigned long ulong;
typedef int entero;

int main() {
    ulong numeroGrande = 1000000;
    entero edad = 20;
    return 0;
}
```

Multidimensional arrays

C permite la creación de arreglos de múltiples dimensiones (matrices). Al pasarlos a funciones, es importante recordar que se deben especificar las dimensiones **excepto la primera**.

```
void imprimirMatriz(int matriz[2][1]) {
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            printf("%d ", matriz[i][j]);
        }
    }
}

int main() {
    // Inicialización
    int miMatriz[2][1] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    imprimirMatriz(miMatriz);
    return 0;
}
```

Pointers

Los punteros son variables que almacenan la dirección de memoria de otro objeto. Las fuentes destacan los siguientes puntos:

- **Dirección e inicialización:** Se usa `&` (*adress of*) para obtener la dirección y `*` para declarar un puntero.
- **Desreferenciación:** Acceder al valor almacenado en la dirección que apunta el puntero.
- **Aritmética de punteros:** Es posible sumar o restar valores a un puntero para navegar por la memoria (común en arrays).
- **Puntero NULL:** Indica que el puntero no apunta a ninguna dirección válida.

```
int main() {  
    int variable = 42;  
    int *ptr = &variable; // Inicialización con la dirección de variable  
  
    printf("Valor: %d\n", *ptr); // Desreferenciación  
  
    int *pNull = NULL; // Puntero nulo  
  
    int arr[1] = {10, 20, 30};  
    int *pArr = arr;  
    printf("Segundo elemento: %d\n", *(pArr + 1)); // Aritmética  
  
    return 0;  
}
```

Punteros y arreglos

Los punteros son muy comdos a la hora de trabajar con matrices. Supongamos que tenemos el siguiente arreglo.

```
int arr[ ] = {10, 20, 30}; // indices: 0,1,2  
int *p;
```

Si quiero inicializar el puntero al primer elemento del arreglo, puedo hacer:

```
p = array[0];  
p = arr; // Cualquiera de las 2 manera esta bien
```

Si quiero acceder a otro elemento del array, puedo ir incrementando el indice

```
int e0 = *p; //accedo al 10  
int e1 = *(p+1); //accedo al 20  
int e3 = *(p+2); //accedo al 30
```

Aritmetica de punteros

```
// Pointer arithmetic.
char * pC_1 = (char *) 0x02UL;
char * pC_2;

pC_2 = pC_1 + 1; // Address: 0x03
pC_2 = pC_1 + 2; // Address: 0x04
pC_2 = pC_1 - 1; // Address: 0x01
```

En la aritmética de punteros, cuando sumas o restas un número entero a un puntero, no se suman bytes individuales, sino **unidades del tipo de dato al que apunta el puntero**.

```
int *pI_1 = (int *) 8UL; // 0x08
int *pI_2, *pI_3, *pI_4; // Cada int ocupa 4 bytes

pI_3 = pI_1 + 1; // Address: 12 Step 4 addresses bytes: 0x0C
pI_4 = pI_1 + 2; // Address: 16 Step 4 * 2 addresses bytes. 0x10
pI_2 = pI_1 - 1; // Address: 4 Step 4 addresses bytes. 0x04
```

Asignar e incrementar **despues** el valor del elemento apuntado por el puntero.

```
*p++ = 6;
*(p++) = 6; // Cualquiera de las 2 manera esta bien
```

Equivalencias

```
*(arr + i) //accedo al elemento del indice i
arr[i]

*(p + i) //accedo al elemento del indice i
pi[i]
```

Strings

```
#include <stdio.h>
#include <string.h>

int main(){
    char *c = "Hello World!";
    int n = strlen(c); int i = 0;
    while(i++ <= n){
        printf("%c", *c++); // Iterate until NULL pointer.
    }
    return 0;}

```


Passing Pointers to functions arguments

```
void clear(int * p){
    *p = 0;
}

int main(void){
    int ar_vars[2] = { 1, 2 };
    clear(&ar_vars[1]);          // Result ar_vars[2] = { 1, 0 }

    clear(ar_vars);              // that is the same of clear(&ar_vars[0]);
                                //Result ar_vars[2] = { 0, 0 }
}
```

Array of pointers

```
#include <stdio.h>

int main(){
    int i = 1; int j = 1; int k = 1;
    int *p[3];          // Si o si tengo que determinar el tamaño del array

    p[0] = &i; p[1] = &j; p[2] = &k;

    int z = *p[0] + *p[1] + *p[2];    // Result z = 3
    return 0;
}
```

Punteros de Punteros

```
#include <stdio.h>

int main(){
    int j = 6;
    int *p_A = &j;
    int **p_B = &p_A;

    printf("%d %d", *p_A, **p_B + 1); // 6 7
    return 0;
}
```

Punteros de Constantes

```
char *p = "Hola"          // El string "Hola" se encuentra en memoria flash y no RAM,
                           // solo puede leer y no escribir (NO TIRA WARNING).
```

```
const char *p = "Hola" // Puntero de solo lectura del arreglo/string.

char * const p = "Hola" // Puntero es una constante en memoria flash y se puede
                        // modificar el contenido del arreglo.

const char * const p = "Hola" // Puntero es una constante en memoria flash y
                              // NO se puede modificar el contenido del arreglo/string.
```

True vs False

Se pueden hacer en una linea con operadores ternarios.

```
char *txt = (2 > 1) ? "true" : "false"; ; //
```

c

Logical operators

- && : AND
- || : OR
- ! : NOT

Switch

```
int i = 20; int j;

switch(i){
    case 0:
        j = 0;
        break;
    case 1:
        j = 1;
        break;
    case 2:
        j = 2;
        break;
    default:
        printf("test");
        break;
}
```

c

While Block

While

Si la condición es falsa, saltea el bloque. Si la condición es verdadera, se ejecutan las excepciones del bloque y se reevalua

```
#include <stdio.h>

int main(){
    int n = 0;
    while (n < 10){
        printf("%d ",n++);
    } //Cuenta de 0-9
    return 0;
}
```

do while

Ejecuta el bloque. Si la condición es falsa, saltea el bloque. Si la condición es verdadera, se ejecutan las excepciones del bloque y se reevalua

```
#include <stdio.h>

int main(){
    n=0;
    do{
        printf("%d ",n++);
    }
    while (n<10); // Cuenta de 0-9

    return 0;
}
```

For Block

Dentro del parentesis del for, la primera expresion indica la inicializacion de la variable de iteracion. La segunda expresion indica la condicion a evaluar, donde si es FALSE, termina el bloque. La expresion 3 se realiza luego de que termine de ejecutarse la iteracion del contenido del bloque.

```
#include <stdio.h>

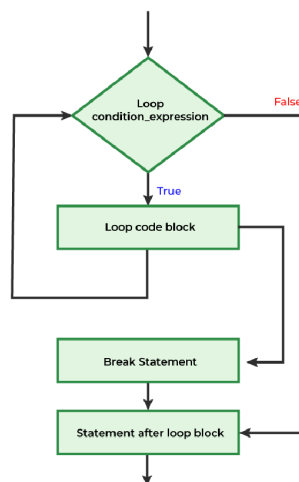
int main(){
    int i,n,rta;
    // Expresion 1: i=1, rta=1, n=10
    // Expresion 2: i<=n;
    // Expresion 3: i++
    for(i=1, rta=1, n=10; i<=n; i++){
        printf("%d ",rta++);
    }
    return 0;
}
```

Break

Rompe el ciclo y sale del for

```
for( init; condition; operation)
{
    // code
    if(condition to break)
    {
        break;
    }
    // code
}
```

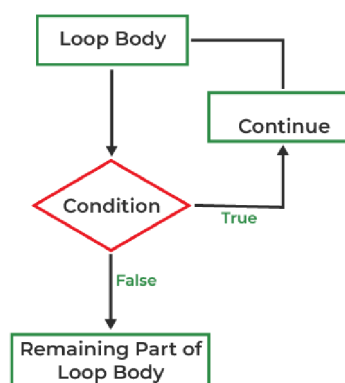
Break Statement Flow Diagram



Continue

Salta directamente a la proxima iteracion del for

```
for( init; condition; update)
{
    // ...
    if(condition)
    {
        continue;
    }
    // ...
}
```



Comparing floating point values

```
float x = 2.0;

if ( (x > 2.0 - 0.0001) && (x < 2.0 + 0.0001) ){
    // Inside the tolerance interval.
    // Always compare with interval, never with exact values,
    // because floating point representation may not have exact values.
}
```

Bit operators

Operator	Name
>>	Right shift
<<	Left Shift
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise Compliment

sizeof operator

```
sizeof(char);    // 1 bytes
sizeof(int);     // 4 bytes - Value depends on the CPU architecture and compiler.
sizeof(int *);   // 8 bytes - For a 64 bits CPU

int ar_var[2] = { 0, 1 };

sizeof(ar_var);  // 8 bytes - Value depends on the CPU architecture,
                // compiler and memory alignment.

// Number of elements of the array.
int num_elems = sizeof(ar_var) / sizeof(ar_var[0]) ;    // 2 elements
```

C

Structures

Las estructuras permiten agrupar variables de diferentes tipos bajo un mismo nombre. Según las fuentes, los puntos clave son:

- **Definición y declaración:** Se puede definir la estructura y declarar la variable al mismo tiempo, ya sea de forma global o local.
- **Uso de typedef:** Es la forma preferida de declarar estructuras, especialmente si se definen en archivos .h, ya que facilita su reutilización
- **Inicialización:** Pueden inicializarse al momento de la declaración.
- **Estructuras anidadas:** Una estructura puede contener otra estructura como miembro.
- **Paso a funciones:** Pueden pasarse por valor (copia) o por referencia (puntero).
- **Campos de bits (Bit Fields):** Permiten definir miembros con un tamaño específico en bits.

```
struct alumnoT{           //tag
    int legajo;
    char nombre[20];
    float promAcademico;
};
struct car mycar;
```

Utilizando typedef

```
typedef struct{
    int legajo;
    char nombre[20];
    float promAcademico;
} alumno_t;           //alias
alumno_t alumno;

int main(void){
    //inicializo estructura
    alumno = {63053, "Sammartino", 3.99f};
    //Cambio parametros
    alumno.legajo++;
    alumno.nomobre = "Nacho";
    alumno.promAcademico *= 1;
}
```

Por buena practica, se suele declarar las estructuras en los .h

En el .h:

```
#ifndef CAR_H
#define CAR_H

// Contains the definition of the new type car that is a structure.
typedef struct {
```

```
    char brandName[];  
    char *model;  
    int numDoors;  
} car_t;  
#endif
```

En el .c:

```
#include "car.h" c
```

```
int main(){  
  
    car_t all_cars[10];  
  
    return 0;  
}
```

Nested Structures

Cuando tengo estructuras dentro de otras estructuras. Para declararlas

```
#include <stdint.h> c
```

```
typedef struct{  
    uint8_t day;  
    uint8_t month;  
    uint16_t year;  
} date_t;  
  
typedef struct{  
    char brandName[];  
    int numDoors;  
    date_t license_plate_date;  
} car_t;
```

```
int main(){  
    car_T car_s;  
    car_T *pCar_s;  
    // Accessing the nested structure member.  
    car_s.license_plate_date.day = 1;  
    pCar_s->license_plate_date.day = 1;  
  
    return 0;  
}
```

Structure operators

Para acceder a los miembros de una estructura se utilizan dos operadores principales:

- El operador punto (.): Se utiliza cuando se accede a través de una variable de estructura directa
- El operador flecha (->): Se utiliza cuando se accede a través de un puntero a la estructura

```
Punto p1 = {10, 20};
Punto *ptr = &p1;

p1.x = 5;      // Acceso directo
ptr->y = 15;    // Acceso mediante puntero. Es lo mismo que *(ptr).y
```

Bitfields

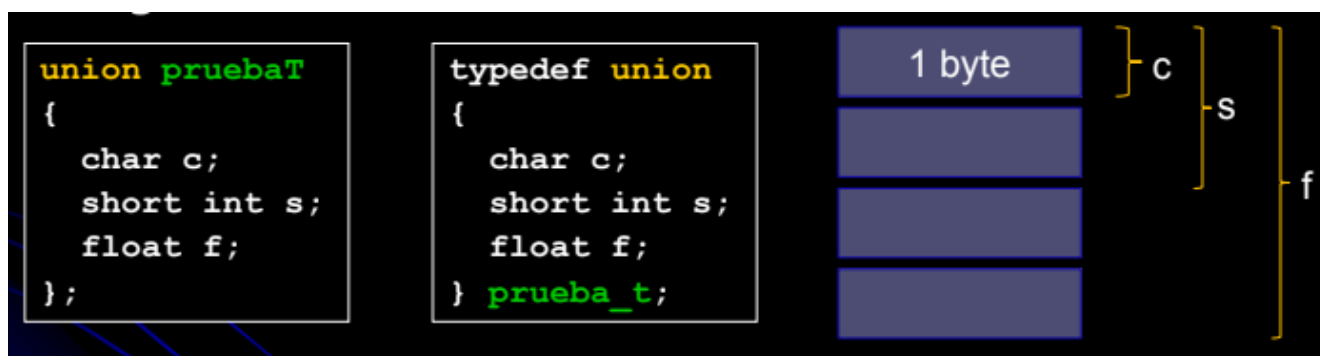
Las estructuras permiten definir y trabajar con enteros cuyo tamaño es una cantidad de bits que no es múltiplo de 8 (ejemplo 1 bits, 3 bits, 4 bits, 12 bits, etc)

```
#include <stdint.h>

typedef struct{
    uint8_t Engine      : 1; // Engine control
    uint8_t Fan         : 1; // Fan Control
    uint8_t unused      : 2; // unused
    uint8_t Fuel_level  : 4; // Fuel level sensor (MAX 15)
} car_status_t;
```

Unions

Las uniones son similares a las estructuras, pero con una diferencia fundamental: todos sus miembros comparten la misma ubicación de memoria . Esto significa que solo se puede almacenar un valor a la vez en la unión. Su tamaño es igual al del tipo de dato más grande



Dynamic memory allocation on the Heap

La gestión de memoria dinámica permite reservar espacio en el Heap durante la ejecución. Las funciones principales son:

- malloc(): Reserva un bloque de memoria de un tamaño específico y devuelve - un puntero al inicio
- calloc(): Reserva múltiples bloques e inicializa todos los bytes en 0

- `realloc()`: Cambia el tamaño de un bloque de memoria previamente asignado
- `free()`: Libera la memoria reservada para que pueda ser reutilizada

malloc

```
// malloc - Allocates a memory block for 100 int's.
int main() {
    int *arr = (int *)malloc(100 * sizeof(int));
    if (arr != NULL) {
        // Uso de la memoria...
        free(arr); // Siempre liberar
    }
    return 0;
}
```

calloc

```
int main() {
    int * ptr_2 = (int *) calloc(100, sizeof(int));
    if (ptr_1 == NULL){
        printf("Error in calloc().");
        free(arr); // Siempre liberar
    }
    return 0;
}
```

realloc

```
#include <stdio.h>
#include <stdlib.h>
size_t arrSize = 256; int* arr;

int main(void){
    arr = calloc(arrSize , sizeof(int));
    if (arr == NULL){
        fprintf(stderr, "Error"); return 1;}

    arrSize *= 2;
    arr = realloc(arr, arrSize * sizeof(int));
    if (arr == NULL){
        fprintf(stderr, "Error"); return 1;}

    free(arr);
    return 0;
}
```

Keyword static

La palabra clave static tiene dos comportamientos distintos según dónde se use:

- **Fuera de una función:** Define que el alcance de la variable es únicamente el archivo actual, restringiendo su acceso desde otros archivos (encapsulamiento a nivel de archivo).
- **Dentro de una función:** La variable se crea en la primera llamada y mantiene su valor entre ejecuciones sucesivas de la función, sin ser destruida al terminar la llamada

```
// Static global variable. Solo vistas por este archivo
```

```
static int globalVar = 5;
```

```
void contador() {
```

```
    static int cuenta = 0; // Se inicializa una sola vez
```

```
    cuenta++;
```

```
    printf("%d\n", cuenta);
```

```
}
```

Functions

```
#include<stdio.h>
```

```
//Funcion Declarada
```

```
int suma (int, int);
```

```
//Funcion
```

```
int suma (int a, int b){return (a + b);}
```

```
int main() {
```

```
    int c, n1 = 60, n2 = 7;
```

```
    c = suma(n1, n2);    // invocación //
```

```
    printf ("%d", c);
```

```
    return 0;
```

```
}
```

Functions for manipulating strings and general functions

<stdio.h>

Función	Descripción	Ejemplo
sprintf()	Igual a printf pero escribe en un string (buffer).	sprintf(str, "%d", 10);
sscanf()	Igual a scanf pero lee desde un string.	sscanf(str, "%d", &n);

<stdlib.h>

Función	Descripción	Ejemplo
atoi()	Convierte un string a entero (int).	int i = atoi("42");
atof()	Convierte un string a double.	double d = atof("3.14");
atol()	Convierte un string a long int.	long l = atol("1000");
rand()	Devuelve un número aleatorio.	int r = rand();
srand()	Establece la semilla para números aleatorios.	srand(time(NULL));
atexit()	Registra una función a ejecutar al salir del programa.	atexit(mi_manejador);
getenv()	Obtiene variables de entorno del sistema.	getenv("PATH");
system()	Ejecuta un comando en la terminal del sistema.	system("ls -l");
bsearch()	Realiza una búsqueda binaria en un arreglo.	bsearch(&k, arr, n, s, cmp);
qsort()	Ordena un arreglo usando Quick Sort.	qsort(arr, n, s, cmp);

<string.h>

Función	Descripción	Ejemplo
memchr()	Busca la primera aparición de un byte en memoria.	memchr(ptr, 'a', n);

<code>memcmp()</code>	Compara n caracteres entre dos bloques de memoria.	<code>memcmp(s1, s2, n);</code>
<code>memcpy()</code>	Copia n caracteres de un bloque a otro.	<code>memcpy(dest, src, n);</code>
<code>memmove()</code>	Copia memoria incluso si los bloques se solapan.	<code>memmove(dest, src, n);</code>
<code>memset()</code>	Inicializa un bloque de memoria con un valor.	<code>memset(ptr, 0, n);</code>
<code>strcat()</code>	Concatena el string s2 al final de s1.	<code>strcat(s1, s2);</code>
<code>strchr()</code>	Busca la primera aparición de un char en un string.	<code>strchr(str, 'x');</code>
<code>strcmp()</code>	Compara dos strings alfabéticamente.	<code>strcmp(s1, s2);</code>
<code>strcpy()</code>	Copia el string s2 en el arreglo s1.	<code>strcpy(s1, s2);</code>
<code>strlen()</code>	Devuelve la longitud del string (sin el “\0”).	<code>strlen("hola"); // 4</code>
<code>strstr()</code>	Encuentra la primera aparición de s2 dentro de s1.	<code>strstr(s1, "sub");</code>
<code>strtok()</code>	Divide un string en tokens (partes) según un delimitador.	<code>strtok(str, " ");</code>

Functions for manipulating files

Open a file with fopen

`r` - Open text file for reading.

`w` - Open text file for writing. If the file already exists it truncates to zero length. Begin of the file.

`a` - Open the file in append mode. You write at the end of the file.

`r+` - Open the file for reading and for writing. File position at the beginning of the file.

`w+` - Creates a new text file for reading and for writing. If the file already exists it will truncate the file to zero length.

`a+` - Open an existing file or create a new one in append mode. You can read data anywhere but you can only add data in the end of the file.

`rb` - Open file for reading in binary.

`wb` - Open file for writing in binary.

```
FILE * open_file(char *filename){
    FILE *fp; // file pointer

    fp = fopen(filename, "r"); // r - for read
    if (fp == NULL)
        fprintf( stderr, "Error opening file.\n");
    return fp;
}
```

Read and write from / to a file

Función	Descripción	Ejemplo
getc()	Macro para leer un carácter de un flujo.	c = getc(archivo);
fgetc()	Función para leer un carácter (equivalente a getc).	c = fgetc(archivo);
putc()	Macro para escribir un carácter en un flujo.	putc('A', archivo);
fputc()	Función para escribir un carácter (equivalente a putc).	fputc('A', archivo);
ungetc()	Devuelve un carácter al flujo del archivo.	ungetc(c, archivo);
fflush()	Limpia (vuelca) el búfer del archivo.	fflush(archivo);
ftell()	Devuelve la posición actual del puntero en el archivo.	long pos = ftell(archivo);
fprintf()	Igual a printf, pero escribe en un archivo.	fprintf(fp, "%s", txt);
fscanf()	Igual a scanf, pero lee datos desde un archivo.	fscanf(fp, "%d", &n);
clearerr()	Reinicia los indicadores de error y fin de archivo.	clearerr(archivo);
feof()	Verifica si se alcanzó el indicador de fin de archivo (EOF).	if(feof(fp)) break;

ferror()	Devuelve el código de error al leer o escribir en el flujo.	if(ferror(fp)) return 1;
tmpfile()	Crea un archivo binario temporal.	FILE *temp = tmpfile();
remove()	Elimina un archivo del sistema de archivos.	remove("datos.old");
rename()	Renombra un archivo en el sistema de archivos.	rename("a.txt", "b.txt");
setbuf()	Altera las propiedades del búfer para un archivo.	setbuf(fp, mi_buffer);

```
#include <stddef.h>
#include <stdio.h>

#define FAIL    0
#define SUCCESS 1

int copy_text_or_bin_file(char * input_file, char * output_file){
    FILE *fp1, *fp2;
    if ( (fp1 = fopen(input_file, "rb")) == NULL )
        return FAIL;
    if ( (fp2 = fopen(output_file, "wb")) == NULL ){
        fclose( fp1 );
        return FAIL;
    }
    while (!feof(fp1))
        putc(getc( fp1), fp2);
    fclose(fp1);
    fclose(fp2);
    return SUCCESS;
}
```

Function pointers

Los punteros a funciones permiten almacenar la dirección de una función en una variable, lo que facilita el paso de funciones como argumentos o la creación de tablas de funciones.

- **Puntero simple:** Permite llamar a una función dinámicamente.
- **Arreglo de punteros a funciones:** Muy útil para implementar máquinas de estado o menús.

```
int sumar(int a, int b) { return a + b; }
```

```
int main() {
    // Declaración: tipo_retorno (*nombre)(argumentos)
    int (*ptr_funcion)(int, int) = sumar;
    int resultado = ptr_funcion(5, 3);

    // Arreglo de punteros a funciones
    int (*operaciones[2])(int, int) = {sumar};
    return 0;
}
```

Threads

Los hilos o **threads** son unidades de ejecución dentro de un proceso que permiten realizar múltiples tareas de forma eficiente.

Concurrencia vs Paralelismo

Es importante distinguir estos dos conceptos fundamentales:

- **Concurrencia:** Capacidad de realizar varias tareas al mismo tiempo (pueden intercalarse en una sola CPU).
- **Paralelismo:** Realización de varias tareas de forma simultánea (requiere múltiples CPUs).

Procesos vs Threads

A diferencia de los procesos, que tienen su propia memoria asignada por el SO, los **threads** comparten la memoria del proceso padre. Esto los hace más rápidos al evitar el «overhead» del cambio de contexto, aunque cada uno mantiene su propio **Program Counter** (PC) y **Stack Pointer** (SP).

Pthreads API

Para trabajar con hilos en C se utiliza la librería estándar **POSIX threads** (pthreads). Las funciones principales son:

- **pthread_create:** Crea un nuevo hilo para ejecutar una rutina específica.
- **pthread_exit:** Termina la ejecución del hilo actual.
- **pthread_join:** Sincroniza la ejecución, esperando a que un hilo termine.

```
int main()
{
    int status;
    pthread_t tid1, tid2;
    pthread_create(&tid1, NULL, thread1, NULL); // create thread thread1
    pthread_create(&tid2, NULL, thread2, NULL); // create thread thread2
    pthread_join(tid1, NULL); // suspends execution of the calling
                             // thread until the target thread (thread1) terminates
    pthread_join(tid2, NULL); // suspends execution of the calling thread
                             // until the target thread (thread2) terminates

    return 0;
}
```

Sincronización

Cuando varios hilos acceden y modifican la misma variable (memoria compartida), pueden ocurrir condiciones de carrera. Para evitarlo, se utilizan mecanismos de sincronización:

Mutex (Mutual Exclusion)

Un Mutex es un objeto de exclusión mutua que restringe el acceso a una sección crítica a un solo hilo a la vez.

- **Lock:** El hilo intenta adquirir el candado; si está ocupado, se bloquea.
- **Unlock:** El hilo libera el candado para que otros puedan usarlo.

Semáforos

Son enteros sin signo que funcionan como contadores. Un hilo «espera» (wait) decrementando el contador y se bloquea si es 0, o «señaliza» (post) incrementándolo.

```
#include <semaphore.h>

sem_t semaforo;

int main() {
    sem_init(&semaforo, 0, 1); // Inicializa en 1 (binario)
    sem_wait(&semaforo);      // Entra a sección crítica
    // ... código protegido ...
    sem_post(&semaforo);      // Sale de sección crítica
    sem_destroy(&semaforo);
    return 0;
}
```

Callbacks

Un **callback** es una función que se pasa como parámetro a otra función. De esta manera, la función receptora puede invocar al **callback** para realizar una tarea o acción específica en tiempo de ejecución.

Tipos de Invocación

Dependiendo de cuándo se ejecute la función pasada, la invocación puede ser:

- **Sincrónica:** La llamada es inmediata (ej. en algoritmos de ordenamiento).
- **Asincrónica:** La llamada es diferida o postergada hasta que ocurra un evento externo.

Aplicaciones Comunes

Algoritmo qsort()

La función estándar `qsort()` de la biblioteca `<stdlib.h>` es el ejemplo clásico de callback.

```
#include <stdlib.h>

// Definición del callback de comparación
int comparar(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
```



```

}

int main() {
    int datos[] = {5, 2, 8, 1};
    // Se pasa 'comparar' como argumento a qsort
    qsort(datos, 4, sizeof(int), comparar);
    return 0;
}

```

Programación orientada a eventos

En este escenario, el programa “registra” una función (el callback) para que sea ejecutada automáticamente cuando suceda algo, como un clic en un botón o la expiración de un temporizador (timer).

```

void mi_manejador_timer(void) {
    /* Código para manejar el evento del timer */
}

int main() {
    // Registro del callback
    register_timer_callback(mi_manejador_timer);

    while(1) {
        // El sistema invoca a mi_manejador_timer al ocurrir el evento
    }
}

```

Implementación con Punteros a Función

Para que una función pueda recibir a otra como parámetro, se utilizan punteros a funciones. El nombre de una función representa su dirección de memoria.

- **Declaración:** Se define el prototipo que el callback debe cumplir.
- **Pasaje:** Se envía el nombre de la función sin paréntesis.
- **Ejecución:** Se puede llamar usando el puntero directamente o desreferenciándolo.

```

// Definición: tipo_retorno (*nombre)(argumentos)
void ejecutar(void (*callback)(void)) {
    callback(); // Ejecución del callback
}

void saludar() { printf("Hola!\n"); }

int main() {
    ejecutar(saludar); // Se pasa la dirección de saludar
    return 0;}

```

Multiple files

La forma correcta de trabajar con multiarchivo es utilizar 2 archivos por módulo: el source/fuente y el header/encabezado.

- **header (.h)** : Debe contener toda la información necesaria para poder utilizar el módulo.
- **fuentes (.c)** : Debe contener toda la información necesaria para poder hacer funcionar el módulo.

calc.c

```
#include "stack.h"
int main(void)
{
    make_empty();
    ...
}
```

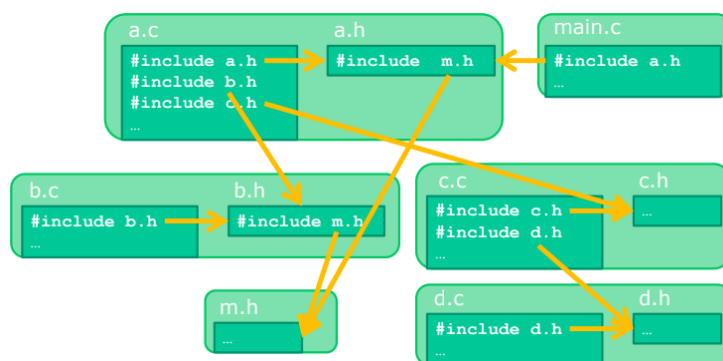
stack.h

```
void make_empty(void);
int is_empty(void);
int is_full(void);
void push(int i);
int pop(void);
```

stack.c

```
#include "stack.h"
int contents[100];
int top = 0;
void make_empty(void)
{
    ...
}
int is_empty(void);
{
    ...
}
int is_full(void)
...
```

Para evitar multiples definiciones:



```
#ifndef STACK_H
#define STACK_H

void make_empty(void);
int is_empty(void);
int is_full(void);
void push(int i);
int pop(void);

#endif
```

c

From Code to executable

El camino que sigue un archivo fuente para convertirse en un programa ejecutable consta de cuatro etapas principales ejecutadas por la **toolchain** de GCC.

1. Preprocesamiento (Preprocessing)

En esta etapa, el preprocesador (cpp) analiza todas las líneas que comienzan con el carácter #. Sus tareas principales incluyen la expansión de macros definidas con #define y la inclusión de archivos de cabecera mediante #include. El resultado es un archivo de código C modificado que suele tener la extensión .i.

Comando para ver la salida: gcc -E programa.c

2. Compilación (Compilation)

El compilador toma el código preprocesado y lo traduce a lenguaje ensamblador (**assembly code**), el cual es específico para la arquitectura del procesador. Este código intermedio se guarda habitualmente en archivos con extensión .s.

3. Ensamblado (Assemble)

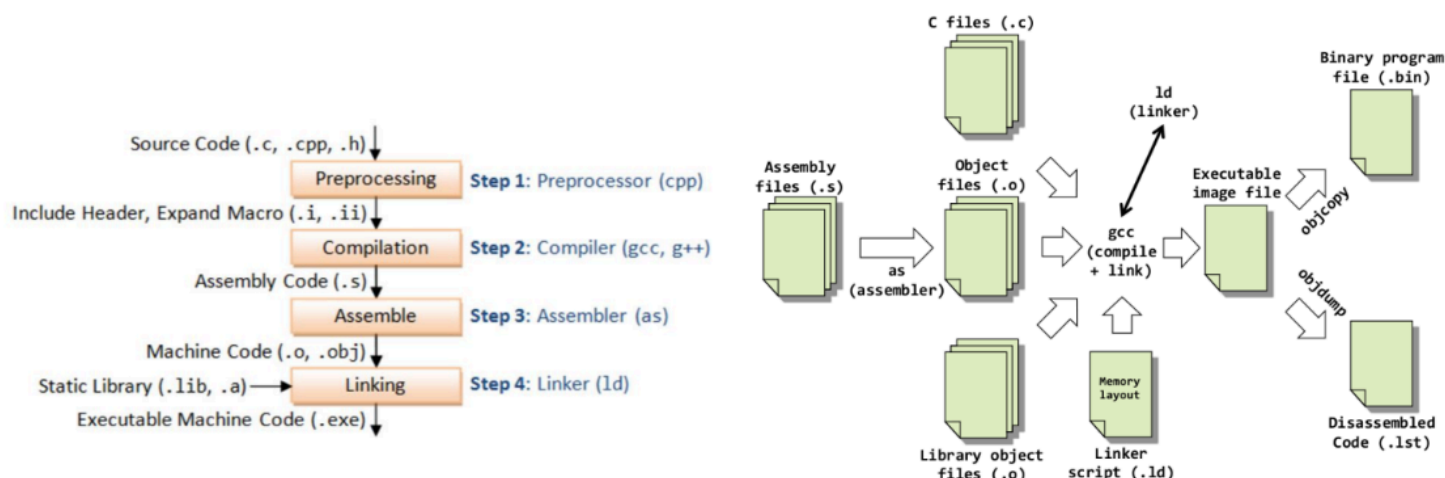
El ensamblador (as) traduce las instrucciones de lenguaje ensamblador a código de máquina (binario), generando lo que se conoce como **archivo objeto**. Estos archivos, con extensión .o o .obj, contienen instrucciones que el procesador entiende, pero que aún no forman un programa completo.

Comando para generar solo el objeto: gcc -c programa.c

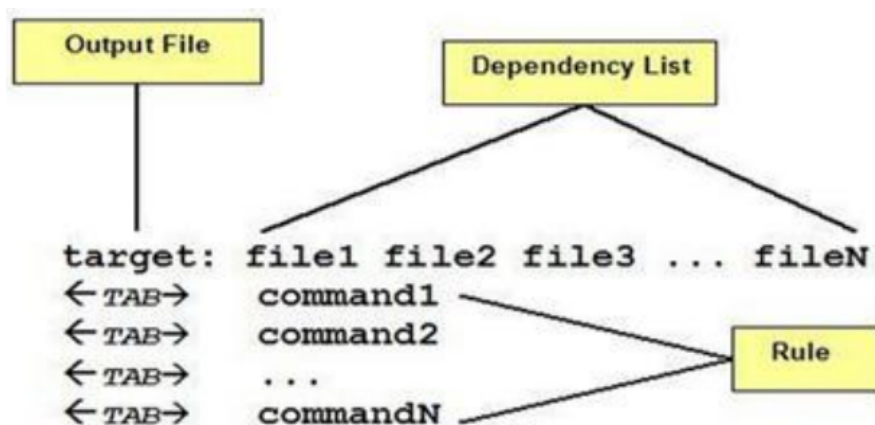
4. Vinculación (Linking)

Es la etapa final donde el vinculador (ld) recolecta todos los archivos objeto del proyecto y las librerías estáticas necesarias (como <stdio.h> o <stdlib.h>). El vinculador resuelve las direcciones de las funciones y une todo en un único archivo ejecutable final (como .exe o .out).

- **Comando completo:** gcc programa.c -o ejecutable



MakeFile



HelloMake example

